

IU DIGITAL RADIO

Guía de implementación del módulo de estado y emisoras

Paso a paso: desarrollo, verificación, publicación e integración

Dato	Asignación
Rol de desarrollo	Luisa Gomez — catálogo de emisoras y estado dinámico
Rama	feature/stations
Revisión funcional	Juan Pablo Gonzalez — interfaz Compose
Integración	Juan Ardila — Pull Request y main
Estado comprobado	Compila y supera 12 pruebas unitarias
Versión de la guía	17/09/2026

Uso del documento. Esta guía enseña el proceso real. Las capturas deben tomarse mientras cada persona ejecuta y explica los pasos; no deben presentarse como evidencia acciones que no realizó.

1. Objetivo y resultado esperado

El módulo de Luisa define la información de las emisoras y las reglas con las que cambia el estado de la pantalla. Al finalizar, la aplicación dispone de un catálogo único, una selección inicial válida, acciones tipadas y una función reductora verificable mediante pruebas unitarias.

- Comprender la diferencia entre modelo, repositorio local, estado de interfaz, acción y reductor.
- Trabajar únicamente en feature/stations sin modificar directamente main.
- Comprobar que los cambios compilan antes de publicarlos.
- Publicar la rama, abrir un Pull Request y permitir que Juan Ardila integre el trabajo en main.

Situación actual. El commit c2a0be6 existe sólo de forma local. La rama feature/stations está un commit por delante de origin/feature/stations; por tanto, todavía no se ha publicado.

2. Alcance y límites del rol

Sí corresponde a Luisa	No corresponde a Luisa
Definir Station y el catálogo inicial.	Diseñar la pantalla principal en Compose.
Definir RadioUiState y las acciones.	Reproducir audio con Media3.
Crear el estado inicial y RadioReducer.	Solicitar permisos o abrir la cámara.
Escribir y ejecutar pruebas unitarias.	Fusionar personalmente cambios en main.
Publicar feature/stations y abrir el PR.	Agregar archivos locales de Android Studio.

Aunque otros módulos consumirán estas clases, Luisa mantiene la responsabilidad inicial sobre sus contratos. Si una integración exige cambiar una acción o un campo, el cambio debe revisarse con quienes consumen ese contrato.

3. Mapa de archivos

Archivo	Función
model/Station.kt	Modelo inmutable de una emisora.
data/StationRepository.kt	Fuente local única del catálogo.
ui/RadioUiState.kt	Estado completo que observará Compose.
ui/RadioAction.kt	Eventos que pueden enviar la pantalla y el hardware.
ui/RadioStateFactory.kt	Construcción coherente del estado inicial.
ui/RadioReducer.kt	Reglas puras para transformar el estado.
StationRepositoryTest.kt	Valida identificadores y campos obligatorios.
RadioReducerTest.kt	Valida selección, reproducción, silencio y cámara.

4. Preparación de la rama

Si Luisa recibe el proyecto por Git, debe trabajar sobre su rama. La rama no está vacía: parte de la arquitectura común aprobada en main y añade únicamente su módulo.

1. Abrir Android Studio y seleccionar el directorio raíz iudigitalradioapp.
2. Abrir la terminal integrada mediante View > Tool Windows > Terminal.
3. Confirmar que Git reconoce el repositorio y revisar el estado.
4. Cambiar a feature/stations. Si todavía no existe localmente, crearla siguiendo origin/feature/stations.
5. Sincronizar Gradle y compilar antes de editar para separar problemas previos de problemas nuevos.

```
git status
git branch --show-current
git switch feature/stations
# Alternativa sólo si la rama no existe localmente:
git switch --track origin/feature/stations
```

Resultado esperado. git branch --show-current debe mostrar feature/stations. No se debe programar directamente en main.

5. Compilación inicial en Android Studio

1. Esperar a que termine Gradle Sync; la barra inferior no debe indicar indexación pendiente.
2. Usar Build > Assemble Project. En versiones recientes puede no aparecer Make Project; Assemble Project cumple la verificación equivalente del módulo.
3. Si aparece Unable to find Gradle tasks to build, abrir el proyecto desde la carpeta raíz que contiene settings.gradle.kts y gradlew.bat, ejecutar Sync Project with Gradle Files y revisar el JDK de Gradle.
4. Usar el JBR incluido con Android Studio cuando se compile desde PowerShell.

```
$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"
.\gradlew.bat assembleDebug
```

6. Implementación explicada

Los siguientes archivos corresponden al código verificado en la rama. Las rutas parten de `app/src/main/java` o `app/src/test/java`, según se indique.

6.1. Modelo Station

Ruta: `app/src/main/java/com/example/iudigitalradioapp/model/Station.kt`

Station representa un dato del dominio: identifica una emisora y reúne los textos y la futura dirección de transmisión.

- `data class` genera `equals`, `hashCode`, `toString` y `copy`, útiles para estado y pruebas.
- `id` debe ser estable y único; la interfaz lo usa para conservar la selección.
- `streamUrl` queda como contrato para el módulo de audio. Las URL serán validadas por Jorge.

```
package com.example.iudigitalradioapp.model

/**
 * Rol responsable en la simulación: Luisa Gomez.
 *
 * Representa una emisora disponible en el catálogo. La interfaz usa sus datos
 * para mostrar la selección y el reproductor utiliza [streamUrl] como fuente.
 */
data class Station(
    val id: String,
    val name: String,
    val description: String,
    val streamUrl: String
)
```

6.2. Repositorio local de emisoras

Ruta: `app/src/main/java/com/example/iudigitalradioapp/data/StationRepository.kt`

El paquete `data` contiene la fuente de datos. `StationRepository` no es una pantalla ni una clase de reproducción: sólo ofrece una colección común para que ningún componente duplique el catálogo.

- `object` crea una única instancia accesible en toda la aplicación.
- `List<Station>` expresa una colección de modelos tipados.
- Las URL vacías son deliberadas hasta que el responsable de audio verifique fuentes HTTPS reproducibles.

```
package com.example.iudigitalradioapp.data

import com.example.iudigitalradioapp.model.Station

/**
 * Rol responsable en la simulación: Luisa Gomez.
 *
 * Mantiene una única colección inicial de emisoras para evitar que cada
 * pantalla defina su propia lista. Las URL se completarán con fuentes HTTPS
 * verificadas durante la integración del reproductor.
 */
object StationRepository {

    val stations: List<Station> = listOf(
        Station(
            id = "wkdu",
            name = "WKDU Philadelphia 91.7 FM",
            description = "Radio universitaria - Drexel University",
            streamUrl = ""
        ),
        Station(
            id = "kexp",
            name = "KEXP",
            description = "Música alternativa e independiente",
            streamUrl = ""
        ),
        Station(
            id = "nucrooze",
            name = "NUCROOZE Radio",
            description = "Jazz, soul y funk",
            streamUrl = ""
        )
    )
}
```

```

        Station(
            id = "groove-salad",
            name = "Groove Salad",
            description = "Ambient y downtempo",
            streamUrl = ""
        ),
        Station(
            id = "indie-pop",
            name = "Indie Pop Rocks!",
            description = "Indie y pop alternativo",
            streamUrl = ""
        )
    )
}

```

6.3. Estado de la interfaz

Ruta: app/src/main/java/com/example/iudigitalradioapp/ui/RadioUiState.kt

RadioUiState reúne todo lo que la pantalla necesita representar. Compose reaccionará cuando se entregue una copia nueva del estado.

- Los valores predeterminados permiten crear un estado vacío de forma segura.
- selectedStationId almacena el identificador; selectedStation calcula el objeto correspondiente.
- capturedPhoto prepara la integración con cámara sin abrirla desde esta clase.

```

package com.example.iudigitalradioapp.ui

import android.graphics.Bitmap
import com.example.iudigitalradioapp.model.Station

/**
 * Rol responsable en la simulación: Luisa Gomez.
 *
 * Reúne los datos observables de la pantalla. Cada cambio produce una copia
 * inmutable para que Compose pueda representar el nuevo estado.
 */
data class RadioUiState(
    val stations: List<Station> = emptyList(),
    val selectedStationId: String? = null,
    val isPlaying: Boolean = false,
    val isMuted: Boolean = false,
    val capturedPhoto: Bitmap? = null
) {
    /** Devuelve la emisora completa que corresponde al identificador activo. */
    val selectedStation: Station?
        get() = stations.firstOrNull { station ->
            station.id == selectedStationId
        }
}

```

6.4. Contrato de acciones

Ruta: app/src/main/java/com/example/iudigitalradioapp/ui/RadioAction.kt

RadioAction enumera las intenciones posibles. Una sealed interface permite que when conozca todas las variantes y exija tratarlas.

- data object se usa cuando la acción no lleva información adicional.
- data class se usa cuando el evento transporta datos, como stationId o Bitmap.
- Definir acciones no ejecuta efectos; RadioRoute conectará cámara, audio y vibración.

```

package com.example.iudigitalradioapp.ui

import android.graphics.Bitmap

/**
 * Rol responsable inicial en la simulación: Luisa Gomez.
 *
 * Enumera las acciones que la pantalla puede enviar al coordinador. Los
 * módulos de UI, cámara y audio consumen este contrato sin duplicar eventos.
 */
sealed interface RadioAction {

    data object Play : RadioAction

    data object Pause : RadioAction
}

```

```

data object ToggleMute : RadioAction

data object OpenCamera : RadioAction

data class SelectStation(
    val stationId: String
) : RadioAction

data class PhotoCaptured(
    val bitmap: Bitmap
) : RadioAction
}

```

6.5. Fábrica del estado inicial

Ruta: app/src/main/java/com/example/iudigitalradioapp/ui/RadioStateFactory.kt

createInitialRadioUiState centraliza el arranque: carga el repositorio y selecciona la primera emisora si existe.

- Recibir stations como parámetro facilita probar listas pequeñas o vacías.
- firstOrNull evita una excepción cuando no hay emisoras.
- El valor predeterminado conecta la función con StationRepository sin impedir pruebas aisladas.

```

package com.example.iudigitalradioapp.ui

import com.example.iudigitalradioapp.data.StationRepository
import com.example.iudigitalradioapp.model.Station

/**
 * Rol responsable en la simulación: Luisa Gomez.
 *
 * Crea un estado coherente al iniciar la aplicación. Recibe la lista como
 * parámetro para permitir pruebas y usa el repositorio como valor predeterminado.
 */
fun createInitialRadioUiState(
    stations: List<Station> = StationRepository.stations
): RadioUiState {
    return RadioUiState(
        stations = stations,
        selectedStationId = stations.firstOrNull()?.id
    )
}

```

6.6. Reductor de estado

Ruta: app/src/main/java/com/example/iudigitalradioapp/ui/RadioReducer.kt

RadioReducer recibe el estado anterior y una acción, y devuelve el siguiente estado. Es una función pura: no reproduce, no vibra y no abre la cámara.

- copy modifica sólo los campos necesarios y conserva el resto.
- Play se ignora si no existe una emisora seleccionada.
- SelectStation valida el identificador y detiene la reproducción mientras integración cambia la fuente.
- OpenCamera no modifica el estado puro; el efecto se ejecutará fuera del reductor.

```

package com.example.iudigitalradioapp.ui

/**
 * Rol responsable en la simulación: Luisa Gomez.
 *
 * Transforma un estado y una acción en un nuevo estado inmutable. No abre la
 * cámara, no reproduce audio y no ejecuta vibración; esos efectos se conectan
 * posteriormente desde RadioRoute.
 */
object RadioReducer {

    fun reduce(
        state: RadioUiState,
        action: RadioAction
    ): RadioUiState {
        return when (action) {
            RadioAction.Play -> {
                if (state.selectedStation == null) {
                    state
                } else {
                    state.copy(isPlaying = true)
                }
            }
            // ...
        }
    }
}

```

```

    }
  }

  RadioAction.Pause -> state.copy(isPlaying = false)

  RadioAction.ToggleMute -> state.copy(isMuted = !state.isMuted)

  RadioAction.OpenCamera -> state

  is RadioAction.SelectStation -> {
    val stationExists = state.stations.any { station ->
      station.id == action.stationId
    }

    if (stationExists) {
      // La reproducción se detiene hasta que integración cargue la nueva fuente.
      state.copy(
        selectedStationId = action.stationId,
        isPlaying = false
      )
    } else {
      state
    }
  }

  is RadioAction.PhotoCaptured -> {
    state.copy(capturedPhoto = action.bitmap)
  }
}
}
}
}

```

7. Pruebas unitarias

Las pruebas describen el comportamiento esperado y permiten detectar regresiones antes de integrar la interfaz. Se ejecutan en la JVM y no requieren emulador.

7.1. Pruebas del repositorio

Ruta: app/src/test/java/com/example/iudigitalradioapp/data/StationRepositoryTest.kt

Estas pruebas comprueban la calidad mínima del catálogo.

- stationIdsAreUnique impide que dos emisoras compartan el mismo identificador.
- stationRequiredFieldsAreNotBlank exige id, nombre y descripción visibles.

```
package com.example.iudigitalradioapp.data

import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

/** Pruebas del módulo asignado al rol simulado de Luisa Gomez. */
class StationRepositoryTest {

    @Test
    fun stationIdsAreUnique() {
        val ids = StationRepository.stations.map { station -> station.id }

        assertEquals(ids.size, ids.distinct().size)
    }

    @Test
    fun stationRequiredFieldsAreNotBlank() {
        val allRequiredFieldsAreValid = StationRepository.stations.all { station ->
            station.id.isNotBlank() &&
            station.name.isNotBlank() &&
            station.description.isNotBlank()
        }

        assertTrue(allRequiredFieldsAreValid)
    }
}
```

7.2. Pruebas del reductor

Ruta: app/src/test/java/com/example/iudigitalradioapp/ui/RadioReducerTest.kt

Estas pruebas ejercitan las transiciones sin depender de Compose ni de hardware.

- Se valida la selección inicial y el catálogo vacío.
- Se valida una selección válida y el rechazo de identificadores desconocidos.
- Se validan Play, Pause, ToggleMute y el carácter puro de OpenCamera.

```
package com.example.iudigitalradioapp.ui

import com.example.iudigitalradioapp.model.Station
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNull
import org.junit.Assert.assertTrue
import org.junit.Test

/** Pruebas del módulo asignado al rol simulado de Luisa Gomez. */
class RadioReducerTest {

    private val stations = listOf(
        Station("station-1", "Emisora 1", "Descripción 1", ""),
        Station("station-2", "Emisora 2", "Descripción 2", "")
    )

    private val initialState = createInitialRadioUiState(stations)

    @Test
    fun initialStateSelectsFirstStation() {
        assertEquals("station-1", initialState.selectedStationId)
        assertEquals("Emisora 1", initialState.selectedStation?.name)
    }
}
```

```

@Test
fun emptyCatalogHasNoSelection() {
    val state = createInitialRadioUiState(emptyList())

    assertNull(state.selectedStationId)
    assertNull(state.selectedStation)
}

@Test
fun selectStationChangesSelectionAndStopsPlayback() {
    val playingState = initialState.copy(isPlaying = true)

    val result = RadioReducer.reduce(
        state = playingState,
        action = RadioAction.SelectStation("station-2")
    )

    assertEquals("station-2", result.selectedStationId)
    assertEquals("Emisora 2", result.selectedStation?.name)
    assertFalse(result.isPlaying)
}

@Test
fun selectUnknownStationKeepsCurrentState() {
    val result = RadioReducer.reduce(
        state = initialState,
        action = RadioAction.SelectStation("unknown")
    )

    assertEquals(initialState, result)
}

@Test
fun playStartsWhenAStationIsSelected() {
    val result = RadioReducer.reduce(initialState, RadioAction.Play)

    assertTrue(result.isPlaying)
}

@Test
fun playDoesNothingWithoutASelectedStation() {
    val stateWithoutSelection = RadioUiState(stations = stations)

    val result = RadioReducer.reduce(stateWithoutSelection, RadioAction.Play)

    assertFalse(result.isPlaying)
}

@Test
fun pauseStopsPlayback() {
    val playingState = initialState.copy(isPlaying = true)

    val result = RadioReducer.reduce(playingState, RadioAction.Pause)

    assertFalse(result.isPlaying)
}

@Test
fun toggleMuteInvertsMutedState() {
    val mutedState = RadioReducer.reduce(initialState, RadioAction.ToggleMute)
    val unmutedState = RadioReducer.reduce(mutedState, RadioAction.ToggleMute)

    assertTrue(mutedState.isMuted)
    assertFalse(unmutedState.isMuted)
}

@Test
fun openCameraDoesNotChangePureUiState() {
    val result = RadioReducer.reduce(initialState, RadioAction.OpenCamera)

    assertEquals(initialState, result)
}
}

```

7.3. Matriz de verificación

Caso	Resultado esperado
Catálogo inicial	Cinco emisoras con identificadores únicos y campos requeridos.
Inicio con lista	La primera emisora queda seleccionada.
Inicio sin lista	No existe selección y no se produce excepción.
Seleccionar emisora válida	Cambia el id seleccionado y se pausa la reproducción.
Seleccionar id desconocido	El estado permanece igual.
Play sin selección	No inicia reproducción.
Play/Pause con selección	El indicador isPlaying cambia correctamente.
Silencio	Cada ToggleMute invierte isMuted.
Abrir cámara	El estado puro permanece intacto.

7.4. Ejecución comprobada

```
$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"
.\gradlew.bat testDebugUnitTest assembleDebug
```

Resultado verificado. BUILD SUCCESSFUL. Se ejecutaron 12 pruebas en total: 2 de StationRepositoryTest, 9 de RadioReducerTest y 1 prueba base del proyecto, con 0 fallos y 0 errores.

8. Revisión antes del commit

Antes de registrar los cambios, Luisa debe leer el diff y confirmar que sólo aparecen archivos de su responsabilidad. En el equipo actual, `.idea/misc.xml` contiene una preferencia local del JDK y no debe incorporarse.

```
git status --short --branch
git diff
git diff --check
```

Comprobación	Qué debe observarse
Rama	feature/stations
Formato	git diff --check sin errores.
Compilación	BUILD SUCCESSFUL.
Archivos locales	<code>.idea/misc.xml</code> fuera del área de preparación.
Datos sensibles	Sin claves, contraseñas ni rutas personales nuevas.

9. Crear el commit de Luisa

Se recomienda agregar rutas específicas y evitar `git add .`, porque este último podría incluir configuraciones locales no relacionadas.

```
git add README.md
git add app/src/main/java/com/example/iudigitalradioapp/model/Station.kt
git add app/src/main/java/com/example/iudigitalradioapp/data/StationRepository.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/RadioUiState.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/RadioAction.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/RadioStateFactory.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/RadioReducer.kt
git add app/src/test/java/com/example/iudigitalradioapp/data/StationRepositoryTest.kt
git add app/src/test/java/com/example/iudigitalradioapp/ui/RadioReducerTest.kt
git diff --cached
git commit -m "Implementa estado y selección de emisoras"
```

En este proyecto. El commit ya fue creado localmente con el identificador corto `c2a0be6`. No debe repetirse. `git status` indica `ahead 1` porque todavía no se ha enviado al remoto.

10. Crear una copia ZIP

El ZIP es un respaldo limpio del contenido registrado en el commit. No sustituye Git, el Pull Request ni la revisión. Al generarse desde HEAD, no incluye `.idea/misc.xml` ni otros cambios sin commit.

```
git archive --format=zip --output ../IU_Digital_Radio_Luisa_Gomez.zip HEAD
```

Guardar el ZIP fuera del repositorio evita que se agregue accidentalmente a Git. Antes de compartirlo, comprobar que contiene `app`, `Gradle`, `README.md` y los archivos Kotlin esperados.

11. Publicar la rama

Este paso se ejecutará únicamente cuando Juan decida que la revisión local terminó. Publicar la rama no modifica main.

```
git switch feature/stations
git status --short --branch
git push origin feature/stations
```

Resultado esperado. GitHub mostrará feature/stations actualizada con el commit de Luisa. La rama main seguirá sin cambios hasta aprobar y fusionar el Pull Request.

12. Abrir el Pull Request

1. Entrar al repositorio en GitHub y seleccionar Compare & pull request.
2. Confirmar base: main y compare: feature/stations.
3. Usar un título concreto: Implementa estado y selección dinámica de emisoras.
4. Describir el alcance, las pruebas ejecutadas y los pendientes de integración.
5. Asignar la revisión a Juan Ardila. Juan Pablo puede revisar que el contrato sea fácil de consumir desde Compose.
6. No fusionar el PR hasta que la compilación y la revisión estén aprobadas.

```
Resumen
- Agrega el catálogo local de emisoras.
- Define RadioUiState y RadioAction.
- Implementa el estado inicial y RadioReducer.
- Agrega pruebas unitarias del repositorio y las transiciones.

Verificación
- ./gradlew.bat testDebugUnitTest assembleDebug
- 12 pruebas, 0 fallos, BUILD SUCCESSFUL.

Pendiente
- Jorge completará y validará las URL HTTPS de reproducción.
- Juan conectará efectos en RadioRoute durante la integración.
```

13. Revisión e integración por Juan Ardila

La integración debe hacerse mediante el Pull Request. Juan revisa el código, ejecuta las pruebas y sólo entonces fusiona en main.

1. Actualizar las referencias remotas y revisar el PR sin escribir sobre la rama de Luisa.
2. Cambiar a feature/stations o crear una rama local de revisión y ejecutar las pruebas.
3. Comprobar que la diferencia contra origin/main sólo contiene los archivos esperados.
4. Aprobar y fusionar el PR en GitHub cuando no existan conflictos ni fallos.
5. Actualizar main local y comprobar nuevamente el proyecto integrado.

```
git fetch origin
git switch feature/stations
git diff --stat origin/main...HEAD
.\gradlew.bat testDebugUnitTest assembleDebug

# Después de fusionar el PR en GitHub:
git switch main
git pull --ff-only origin main
.\gradlew.bat testDebugUnitTest assembleDebug
```

Responsabilidad. Luisa publica y explica su trabajo; Juan Ardila acepta la integración. Esta separación permite rastrear autoría, revisión y calidad sin que todos escriban directamente en main.

14. Cómo continúa Juan Pablo

Juan Pablo no debe recoger directamente la rama de Luisa. Debe esperar a que el PR quede integrado en main y después actualizar feature/ui-compose desde main. Así recibe una versión revisada y se evita crear dependencias entre ramas personales.

```
git fetch origin
git switch feature/ui-compose
git merge origin/main
# Resolver conflictos, si aparecen, y comprobar:
.\gradlew.bat testDebugUnitTest assembleDebug
```

Después podrá utilizar Station, RadioUiState y RadioAction para construir la pantalla Compose. La integración de efectos reales seguirá perteneciendo a RadioRoute y a los responsables de hardware y audio.

15. Evidencias sugeridas para el día de trabajo

Las capturas son evidencia de lo realizado y comprendido, no sólo imágenes del resultado final. Deben numerarse, incluir una breve explicación y evitar datos personales innecesarios.

N.º	Captura	Explicación sugerida
1	Terminal mostrando feature/stations.	Se verificó que el desarrollo se realizó en la rama asignada.
2	Árbol de paquetes model, data y ui.	Se organizó el código por responsabilidades.
3	StationRepository.kt.	Se creó un catálogo único con identificadores y descripciones.
4	RadioUiState.kt y RadioAction.kt.	Se definieron el estado observable y los eventos del sistema.
5	RadioReducer.kt.	Se implementaron transiciones inmutables mediante copy.
6	Ventana de pruebas exitosas.	Las reglas se validaron con pruebas unitarias.
7	BUILD SUCCESSFUL.	La rama compila antes de ser publicada.
8	Commit c2a0be6 o historial Git.	Los cambios quedaron agrupados en un commit descriptivo.
9	Pull Request abierto.	Se solicitó revisión antes de modificar main.
10	PR fusionado y main actualizado.	Juan Ardila integró una versión revisada y compilable.

Pie de figura recomendado. Figura 1. Verificación de la rama feature/stations antes de implementar el catálogo.
Nota. Elaboración propia (captura de Android Studio y Git).

16. Lista de cierre

- ☐ Estoy en feature/stations y entiendo por qué no trabajo directamente en main.
- ☐ Puedo explicar Station, StationRepository, RadioUiState, RadioAction y RadioReducer.
- ☐ Ejecuté testDebugUnitTest y assembleDebug con resultado exitoso.
- ☐ Revisé el diff y excluí .idea/misc.xml y archivos personales.
- ☐ El commit tiene un mensaje descriptivo y la rama fue publicada sólo después de revisar.
- ☐ El Pull Request compara feature/stations contra main.
- ☐ Juan Ardila revisó, fusionó y volvió a comprobar main.
- ☐ Juan Pablo actualizó feature/ui-compose desde main, no directamente desde la rama de Luisa.

17. Preguntas para comprobar el aprendizaje

- ¿Por qué Station es una data class y StationRepository es un object?
- ¿Qué ventaja tiene guardar selectedStationId en lugar de duplicar el objeto seleccionado?
- ¿Por qué RadioReducer no debe abrir la cámara ni iniciar Media3?

- ¿Qué protege firstOrNull cuando el catálogo está vacío?
- ¿Por qué se detiene isPlaying al cambiar de emisora?
- ¿Qué diferencia existe entre commit, push, Pull Request y merge?

18. Referencias técnicas

Android Developers. (s. f.). State and Jetpack Compose.
<https://developer.android.com/develop/ui/compose/state>

Android Developers. (s. f.). Guide to app architecture: Compose architecture.
<https://developer.android.com/develop/ui/compose/architecture>

IU Digital. (s. f.). Evidencia de aprendizaje 3: Aplicación móvil Android IU Digital Radio [Guía de orientación].

Nota: las referencias se presentan como apoyo técnico. En el informe académico final deben ajustarse a los datos bibliográficos completos disponibles en el aula virtual y a las normas APA 7.

19. Resumen del flujo

Responsable	Acción	Resultado
Luisa Gomez	Implementa y prueba en feature/stations.	Commit local revisado.
Luisa Gomez	Publica la rama y abre el Pull Request.	Cambio visible para revisión.
Juan Ardila	Revisa, compila y fusiona.	main contiene el módulo aprobado.
Juan Pablo Gonzalez	Actualiza feature/ui-compose desde main.	La UI consume contratos estables.